# CS 162 Fall 2025 Homework Companion

Handout Review Report & Local Testing Handbook (student0 mirror)

Compiled June 2026 — consolidates the repository's former README/STUDY/TESTING/

2026-06-10

# Contents

**Preface** **4**
0.1 How to use this document . . . 4
0.2 Environment assumed throughout . . . 4
0.3 Conventions . . . 5

**I Fall 2025 Handout Review** **6**

**1 Review methodology** **7**
1.1 Why the Wayback Machine . . . 7
1.2 Pipeline . . . 7
1.3 Snapshot quality notes . . . 7
1.4 What “review, report-only” means here . . . 8
1.5 Lineup verification . . . 8

**2 HW 0: Introduction (`hw-intro`)** **9**
2.1 Official assignment (Fall 2025) . . . 9
2.2 Starter map (`ORIG`) . . . 9
2.3 Environment notes for local work . . . 10
2.4 Review findings (former in-repo docs) . . . 10
2.5 Test contract . . . 10

**3 HW 1: List (`hw-list`)** **11**
3.1 Official assignment (Fall 2025) . . . 11
3.2 Starter map (`ORIG`) . . . 11
3.3 Review findings (former in-repo docs) . . . 12
3.4 Test contract . . . 12

**4 HW 2: Shell (`hw-shell`)** **13**
4.1 Official assignment (Fall 2025) . . . 13
4.2 Starter map (`ORIG`) . . . 13
4.3 Review findings (former in-repo docs) . . . 14
4.4 Test contract . . . 14

**5 HW 3: HTTP (`hw-http`)** **15**
5.1 Official assignment (Fall 2025) . . . 15
5.2 Starter map (`ORIG`) . . . 15
5.3 Review findings (former in-repo docs) . . . 16
5.4 Test contract . . . 16

**6 HW 3: Rust HTTP Server (`hw-http-rs`)** **17**
6.1 Official assignment (Fall 2025) . . . 17
6.2 Starter map (`ORIG`) . . . 17

6.3 Review findings (former in-repo docs) . . . 18
6.4 Test contract . . . 18

**7 HW 4: Memory (`hw-memory`) 19**
7.1 Official assignment (Fall 2025) . . . 19
7.2 Review findings (former in-repo docs) . . . 20
7.3 Test contract . . . 20

**8 HW 5: Map Reduce (`hw-map-reduce`) 21**
8.1 Official assignment (Fall 2025) . . . 21
8.2 Review findings (former in-repo docs) . . . 22
8.3 Test contract . . . 22

**9 HW 5: Map Reduce, Rust (`hw-map-reduce-rs`) 23**
9.1 Official assignment (Fall 2025) . . . 23
9.2 Review findings (former in-repo docs) . . . 24
9.3 Test contract . . . 24

**10 RPC Lab (`lab-rpc`) 25**
10.1 Official assignment (Fall 2025) . . . 25
10.2 Review findings (former in-repo docs) . . . 25
10.3 Test contract . . . 26

**11 gRPC Lab (`lab-grpc-rs`) 27**
11.1 Official assignment (Fall 2025) . . . 27
11.2 Review findings (former in-repo docs) . . . 27
11.3 Test contract . . . 28

**12 Project Pregame (`proj-pregame`) 29**
12.1 Official assignment (Fall 2025) . . . 29
12.2 Starter map (`ORIG`) . . . 29
12.3 Review findings (former in-repo docs) . . . 29
12.4 Test contract . . . 30

## II Local Testing Handbook 31

**13 Testing fundamentals 32**
13.1 What a test is . . . 32
13.2 Fixtures and golden files . . . 32
13.3 Timeouts, servers, and ports . . . 33
13.4 Concurrency . . . 33
13.5 Memory . . . 33
13.6 Signals need a pty . . . 33
13.7 Honest skipping . . . 33
13.8 Reading a failure, and TDD order . . . 34
13.9 Smells . . . 34

**14 Framework reference 35**
14.1 Layout . . . 35
14.2 Running . . . 35
14.3 Outcome protocol . . . 36
14.4 Writing a case . . . 36
14.5 `tests/lib.sh` API . . . 36
14.6 Rust crates . . . 37
14.7 Pintos pieces . . . 37

14.8 Using the REPORT.md templates . . . 37

**15 Per-assignment test recipes** **38**
15.1 hw-intro . . . 38
15.2 hw-list . . . 38
15.3 hw-shell . . . 38
15.4 hw-http . . . 39
15.5 hw-http-rs . . . 39
15.6 hw-memory . . . 39
15.7 hw-map-reduce . . . 39
15.8 hw-map-reduce-rs . . . 40
15.9 lab-rpc . . . 40
15.10 lab-grpc-rs . . . 40
15.11 proj-pregame . . . 40

**III Appendices** **41**

**A Self-check questions and answers** **42**
A.1 hw-intro . . . 42
A.2 hw-list . . . 42
A.3 hw-shell . . . 43
A.4 hw-http . . . 43
A.5 hw-http-rs . . . 44
A.6 hw-memory . . . 44
A.7 hw-map-reduce (C) . . . 45
A.8 hw-map-reduce-rs . . . 45
A.9 lab-rpc . . . 45
A.10 lab-grpc-rs . . . 46
A.11 proj-pregame . . . 46

**B Validation record** **48**
B.1 Prior validation (2026-05-10, absorbed from the former `validation-report.md`) . . . 48
B.2 Final framework run (2026-06-10, this repository state) . . . 48

**C Wayback snapshot index** **50**
C.1 hw-http-rs . . . 50
C.2 hw-http . . . 50
C.3 hw-intro . . . 50
C.4 hw-list . . . 51
C.5 hw-map-reduce-rs . . . 51
C.6 hw-map-reduce . . . 52
C.7 hw-memory . . . 52
C.8 hw-shell . . . 52
C.9 lab-grpc-rs . . . 53
C.10 lab-rpc . . . 53
C.11 proj-pregame . . . 53

# Preface

This document is the single consolidated companion for the `student0` repository — a fork of Berkeley CS 162's official Fall 2025 homework starter (`Berkeley-CS162/student0`, frozen locally at commit `895ecb5`, "lab-mapreduce", 2025-11-25). It replaces and absorbs all previously added loose documentation (per-assignment `README.md`, `STUDY.md`, `TESTING.md`, and the `docs/` guides), so that the repository's only additions over the pristine official starter are:

1. **a local test framework** — `tests/lib.sh`, `tests/runner.sh`, a thin `tests/run.sh` per assignment, and per-assignment test cases you are meant to extend;
2. **one `REPORT.md` lab-report template per assignment** (markdown, for you to fill in);
3. **this PDF.**

## 0.1 How to use this document

- **Part I** is a review report: for each of the 11 assignments it digests the official Fall 2025 handout (retrieved from the Internet Archive's Wayback Machine, with snapshot citations), maps the handout's tasks onto the starter skeleton, and records where the repository's former documentation was confirmed, unsupported, or plainly wrong. Read a chapter *before* starting that assignment — the "Review findings" tables double as a list of traps.
- **Part II** is the testing handbook: the testing techniques in one place (Chapter 13), the framework reference (Chapter 14), and per-assignment test recipes with a roadmap from the shipped starter cases to a complete, autograder-shaped suite of your own (Chapter 15).
- **Appendices**: curated self-check questions with answers (A), the validation record for this workspace (B), and the full Wayback snapshot index backing every citation (C).

## 0.2 Environment assumed throughout

| Item | Value |
|---|---|
| Host | WSL2 Ubuntu, x86-64 |
| C toolchain | gcc + make (+ gcc multilib for Pintos i386 builds) |
| Rust | rustup toolchain per each crate's `rust-toolchain.toml` (cargo ≥ 1.7x) |
| SunRPC | `rpcgen`, `libtirpc-dev`, `libglib2.0-dev`, `rpcbind` service |
| gRPC | CMake ≥ 3.20 (for `tonic-build`) |
| Pintos | `qemu-system-i386`, `perl` |
| PDF tooling | pandoc + XeLaTeX (used to produce this document) |

The course's own environment is the CS 162 Docker workspace with `i386-*` wrapper tools; differences that matter are flagged per assignment.

## 0.3 Conventions

- `ORIG` means the pristine official starter at commit `895ecb5`.
- Citations like *(Wayback 20251110134217)* give the 14-digit snapshot timestamp; resolve as `https://web.archive.org/web/<timestamp>/<original-url>` — the full index is Appendix C.
- Verdicts in the claims-audit tables: **Confirmed** (matches the official handout), **Unsupported** (invented; no basis in the handout), **Contradicts** (conflicts with the handout — following it would have cost points), **Gap** (an official, grading-relevant requirement the former docs failed to mention).

# Part I

# Fall 2025 Handout Review

# Chapter 1

# Review methodology

## 1.1 Why the Wayback Machine

`cs162.org` always shows the *current* semester. This repository's starter is the **Fall 2025** release (the fork's staff commits run 2025-08-26 … 2025-11-25 and merge `Berkeley-CS162/student0`), so the live site — already serving Spring 2026, which among other things added a new `hw-kvsrv` / `hw-kvsrv-rs` Raft assignment that does not exist in Fall 2025 — is the wrong baseline. All handout content in Part I was therefore retrieved from the Internet Archive.

## 1.2 Pipeline

1. **Enumerate.** For each assignment site (`cs162.org/static/hw/<name>/`, plus `static/proj/proj-pregame/`), query the CDX API for every captured URL between 2024-01 and 2026-06, keep status-200 `text/html` rows, and drop assets.
2. **Select.** Per page, prefer the capture inside the Fall 2025 window (2025-08-01 … 2026-01-31) closest to 2025-10-15; otherwise take the nearest capture outside the window and **annotate it**. 115 pages were fetched; 109 were unique handout pages after removing CDX prefix collisions (`hw-http*` also matches `hw-http-rs`).
3. **Fetch raw.** Use the `id_` modifier (`web.archive.org/web/<ts>id_/<url>`) to get the original bytes without the Wayback toolbar; gunzip the ~60% of captures stored gzip-encoded.
4. **Extract.** Slice the `just-the-docs <div id="main-content">` region and convert to markdown with pandoc.
5. **Compare.** For each assignment: handout tasks → `ORIG` skeleton files (verified with `git show 895ecb5:<path>`), build/test commands, autograder test lists, and a claims audit of the repository's former README/STUDY/ TESTING content.

## 1.3 Snapshot quality notes

- **hw-map-reduce-rs** is thinly archived inside the Fall 2025 window (3 pages). Its task pages are cited from complete April 2025 (Sp25) captures; the Fall 2025 index page (Wayback 20251110141540) links the identical page set, and several pages carry Fall-2025 "revisit" records (captures byte- identical to the cited ones). Treat those citations as Sp25 text whose Fall 2025 presence is confirmed but whose Fall 2025 wording is inferred.
- A handful of pages elsewhere (hw-memory library-functions/sbrk-intro, hw-http background/tasks index, proj-pregame tasks/debug, lab-rpc protocol-specification, two hw-shell index pages, hw-list pthread) are cited from adjacent Apr-2025 or Jan-2026 captures for the same reason — each is marked in Appendix C.

- The five `hw-intro/docs/setup/{vm,vm-config,virtualbox-setup,code-vm, cloud-setup}` pages exist only as 2024 captures and are **absent from the Fall 2025 setup navigation** (Docker-workspace era); they were excluded as legacy pages.

## 1.4 What "review, report-only" means here

Part I documents discrepancies; it does not patch the starter. The pristine `ORIG` files — including intentional starter breakage such as `hw-intro/words` and `hw-memory/mm_alloc` failing to compile under `-Werror` until you write the missing code, and `hw-map-reduce-rs`'s `todo!("Pick an input file")` that breaks a full `cargo test` — are part of the assignments and stay byte-identical to the official release.

## 1.5 Lineup verification

Fall 2025 individual-work lineup, all present in `ORIG` and all reviewed here: `hw-intro`, `hw-list`, `hw-shell`, `hw-http`, `hw-http-rs`, `hw-memory`, `hw-map-reduce`, `hw-map-reduce-rs`, `lab-rpc`, `lab-grpc-rs`, `proj-pregame`. No Fall 2025 individual assignment is missing from the repository, and the repository contains no assignment that Fall 2025 lacked. (The Raft-based `hw-kvsrv`/`hw-kvsrv-rs` appear on cs162.org only from Spring 2026 and are intentionally out of scope.)

# Chapter 2

# HW 0: Introduction (`hw-intro`)

## 2.1 Official assignment (Fall 2025)

Orientation homework: course tooling (Docker workspace, GitHub + autograder, git/make/man/gdb/tmux/vim
then four graded sections.

1. **Counting words** *(Wayback 20251215163710)* — implement `words` under `words/`:
   - *Total count*: `./words FILE...` prints `The total number of words is: N`; multiple files accumulate; with no file arguments read stdin (`./words < words.txt`, `printf hi | ./words` → 1). A **word is a contiguous alphabetic run of length > 1**, folded to lower case (`words.txt` fixture: 6 words — the trailing `a` does not count). Implement `num_words()` plus the `main.c` plumbing.
   - *Frequency count*: `./words -f` prints one `<count> <word>` line per unique word — count ascending, alphabetical tiebreak, single space (verified against the raw snapshot HTML: no tabs). `wordcount_sort` is provided precompiled (`wc_sort.o`); you write `wordcount_less` in `main.c` and the whole linked-list API in `words/word_count.c`. The handout warns that `wordcount_sort` may internally call your `init_words`.
   - *Error handling*: check `fopen`/`malloc` results; missing files must fail cleanly (the handout shows the patterns). Avoid `rewind`/`fseek` (they break stdin support).
2. **User limits** *(Wayback 20251215163712)* — `limits.c`: print the **soft** limits for stack size, process count, and open file descriptors via `getrlimit`. The starter prints zeros. The Docker workspace may show a negative process limit ("this is fine").
3. **GDB basics** *(Wayback 20251215163642)* — drive `map` under `bash i386-gdb-map.sh`; the bold questions (address of `argv`, address of `recur`, register-print command, return address into `main`, instructions implementing `return 0`) are **submitted to Gradescope**.
4. **From source code to executable** *(Wayback 20251215163710)* — the `i386-gcc -m32` pipeline (`-E`, `-S`, `-c`), `objdump` exploration of `map.o`/`recurse.S`/`printer.c`; **10 written Gradescope questions**.

Submission *(Wayback 20251215163710)*: `git add words/main.c words/word_count.c limits.c Makefile` → push → autograder email; written answers go to Gradescope only.

## 2.2 Starter map (`ORIG`)

| Path | Role |
|---|---|
| `Makefile` | builds `map`, `limits`; auto-selects `i386-gcc` when present, else plain `gcc` |

| Path | Role |
|---|---|
| `map.c`, `recurse.c`, `printer.c` | GDB/executable-section subjects (provided) |
| `limits.c` | prints the three lines with `0L` placeholders — your task |
| `i386-gdb-map.sh` | pinned to `/home/workspace/code/personal/hw-intro`; refuses to run elsewhere |
| `words/Makefile` | `-Wall -Werror -g3`; links `wc_sort.o` (ARM variant auto-selected) |
| `words/main.c`, `words/word_count.c\|h` | your implementation surface |
| `words/words.txt` | byte-identical to the handout example fixture |
| `words/gutenberg/` | larger inputs |

## 2.3 Environment notes for local work

The course environment wraps everything in `i386-*` tools (qemu-i386 user emulation, so results match the Pintos architecture). Locally, `ORIG`'s Makefile falls back to native `gcc`, and the binaries run natively; `i386-gdb-map.sh` will refuse to run outside the Docker workspace path — the GDB walkthrough must be done in the course workspace (or adapted knowingly). The pristine `words/` does **not** compile (`-Werror` versus the starter's unused variable) until you start the assignment — the framework's `13-words-starter-build` case reports this as an honest SKIP.

## 2.4 Review findings (former in-repo docs)

| Former claim | Verdict |
|---|---|
| Task list: counting words / user limits / gdb / executable | **Confirmed** |
| Frequency output shown as tab-separated (`3\thello`) in the testing guide | **Contradicts** — handout examples use `<count><space><word>` |
| "Word" definition stated without the length-> 1 rule | **Gap** — single-letter tokens must not count |
| No mention of `i386-exec`/`i386-gcc` wrappers | **Gap** — the handout's run instructions differ from native runs |
| No mention of the Gradescope written questions (gdb + executable, 10+5 bold questions) | **Gap** — these are graded deliverables |
| `-Werror` starter build failure of `words/` | **Confirmed** (and now encoded as an honest SKIP) |

## 2.5 Test contract

Framework cases: smoke `10-build-limits`, `11-limits-prints-three-lines`, `12-map-prints-addresses`, `13-words-starter-build`; gated `full/20..25` (handout examples verbatim: totals 6/12, stdin, the exact 5-line frequency block, missing-file failure, non-zero soft limits); TODO stubs `full/40..42` (length-rule fixture, case folding, `ulimit` cross-check). Recipes: Chapter 15.

# Chapter 3

# HW 1: List (`hw-list`)

## 3.1 Official assignment (Fall 2025)

Threads and processes from userspace, plus the Pintos intrusive list — prep for Project Userprog. `make` produces four binaries: `pthread`, `words`, `pwords`, `lwords`; none of them needs a `-f` flag (all word binaries run in frequency mode) *(Wayback 20251008233309)*.

1. **pthread — analysis only** *(Wayback 20260131004901; Fall-2025 revisit record)*. `pthread.c` is complete and stamped "You may NOT modify this file". Run it repeatedly and answer five Gradescope questions (output determinism, shared stacks?, shared globals?, what `void* threadid` really is, what happens with many threads and no joins).
2. **lwords** *(Wayback 20251202115726)* — implement `word_count_l.c` against the `word_count.h` API **using the Pintos list API** (the `PINTOS_LIST` macro changes the struct definitions; read `list.h`'s big docstring first). Linked with the provided `lwords.o` driver, `lwords` must behave exactly like `words`. `wordcount_sort` must work with **any** comparator (`less_word`, not just `less_count`).
3. **pwords** *(Wayback 20251202115733)* — write the driver `pwords.c` (one thread per input file) and `word_count_p.c` (same API, synchronized). Hard requirements: no materializing intermediate results to files, and **fine-grained synchronization** — "it is unacceptable to use a global lock around the call to `count_words` … We reserve the right to deduct points from such implementations." Only access to the shared word-count list is serialized; file reads proceed concurrently. The `gutenberg/` inputs exist to surface subtle races by comparing `pwords` to `words`. Two Gradescope performance questions (lwords vs pwords with `time`; when each wins).

Submission: push (no binaries — `make clean` first); written answers → Gradescope *(Wayback 20251202115733)*.

## 3.2 Starter map (`ORIG`)

| Path | Role |
|---|---|
| `list.{h,c}` | Pintos list, **must not modify** |
| `word_count.h` | the API + structs, **must use** |
| `word_count_l.c` (6 TODOs), `word_count_p.c` (6 TODOs), `pwords.c` (1 TODO) | your implementation surface |
| `pthread.c` | provided complete (50 lines, do-not-modify header) |

| Path | Role |
|---|---|
| `words.o`, `word_count.o`, `word_helpers.o`, `lwords.o` (+ `_arm` variants) | precompiled oracle + drivers |
| `gutenberg/` | race-surfacing dataset |

The reference `words` prints `%8d\t%s` lines (right-aligned count, tab) — parity tests should diff against it rather than hard-coding a format.

## 3.3 Review findings (former in-repo docs)

| Former claim | Verdict |
|---|---|
| `pthread.c` presented as a coding task (“TODO: spawn N threads, print stack addresses, join”) | **Contradicts** — the handout says “you won’t be writing any code in `pthread.c`”, and the file forbids modification |
| Fine-grained-synchronization requirement and the explicit global-lock prohibition | **Gap** — grading-relevant, was absent |
| Gradescope questions (pthread ×5, performance ×2) | **Gap** |
| lwords/pwords/words interface description | **Confirmed** |
| STUDY content on races, joins, intrusive lists | Mostly **Confirmed**; kept (curated) in Appendix A |

## 3.4 Test contract

Smoke `10-build-all`, `11-pthread-runs`, `12-words-reference-output`; gated `full/20..23` (lwords/pwords parity vs the oracle, single/multi-file, gutenberg) and `full/30-pwords-race-stress` (25 iterations); TODO stubs `full/40..42` (fine-grained-concurrency evidence, helgrind, generic- comparator driver). Recipes: Chapter 15.

# Chapter 4

# HW 2: Shell (`hw-shell`)

## 4.1 Official assignment (Fall 2025)

Build a bash-like shell on the provided tokenizer + built-in dispatcher *(index: Wayback 20251110134217; task pages 20251106…)*.

1. **Directory commands** — add built-ins `pwd` and `cd <dir>` to the dispatcher (`?` and `exit` are pre-wired); `chdir`/`getcwd`.
2. **Program execution** — non-built-ins: `fork`, an `exec`-family call in the child, parent `wait`s. At this stage the first word is a **full path**. Tokenization comes from `tokenizer.h` — no extra parsing features needed. **"You may not use `execvp`."**
3. **Path resolution** — walk `$PATH` yourself (`strtok_r` hint) and run the first match via `execv`; full paths must still work. The handout repeats the ban with teeth: **"the autograder looks for `execvp`, and you won't receive a grade if that word is found."**
4. **Redirection** — exactly `[p] > [file]` and `[p] < [file]`. **Not required:** `>>` append, stderr redirection, redirection of built-ins. Tokens around `<`/`>` are guaranteed space-separated and must not reach the program's argv. The handout ships a worked example (`/usr/bin/wc < test_redirection_in > test_redirection_out` on `i love pintos and cs162` → `1 5 24`-shaped output) and explains the classic symptom of a broken `<` (the filename appearing after the numbers).
5. **Pipes** — `A | B | C …` with an unbounded stage count. The page includes a full fd walk-through; the load-bearing rule: every child *and* the shell must close all unused pipe fds or the pipeline hangs.
6. **Signal handling** — each spawned program goes in its **own process group**; the foreground process group is handed the terminal (`tcsetpgrp`); stopping signals affect only the foreground job; the shell ignores most job-control signals itself (children get defaults back) and **must survive Ctrl-C**.
7. **Extra credit (1.5 pts, post-curve)** — background processes (`&`) and `fg`/`bg` switching; staff support explicitly limited.

Testing *(Wayback 20251106180552)*: the starter ships the staff script `test.sh` ("run `bash test.sh`", "feel free to add onto it"). Submission: push → autograder; no debug prints.

## 4.2 Starter map (`ORIG`)

`shell.c` (dispatcher, `cmd_exit`/`cmd_help`, `lookup`, REPL loop), `tokenizer.{c,h}`, `Makefile`, `test.sh`, `.gitignore` — complete and matching the handout's description.

## 4.3 Review findings (former in-repo docs)

| Former claim | Verdict |
|---|---|
| Section/task list (dir cmds, exec, path, redirection, pipes, signals, EC) | **Confirmed** |
| `execvp` listed among the exec calls to use | **Contradicts** — grep-banned by the autograder; would zero the grade |
| `>>` append described as required | **Contradicts** — explicitly not required |
| Signal-handling description (process groups, tcsetpgrp, ignore-in-shell) | **Confirmed** |
| Tokenizer-quoting unit-test advice | **Unsupported** — quoting behavior isn't part of the handout contract; harmless but not graded |

## 4.4 Test contract

Smoke `10-build`, `11-eof-no-hang`, `12-unknown-command-no-crash`, and `13-no-execvp-static` (mirrors the autograder's grep so you discover the ban locally). Gated `full/20..29` implement the handout's own examples (pwd, cd, absolute-path exec, PATH lookup, `>`/`<`, 2- and 3-stage pipes, shell-survives-failure, the `wc` redirection example). TODO stubs `full/40..42` (pty signal tests — recipe in Chapter 13's pty section) and `full/50..51` (EC). Recipes: Chapter 15.

# Chapter 5

# HW 3: HTTP (`hw-http`)

## 5.1 Official assignment (Fall 2025)

An HTTP/1.0 GET server with four concurrency variants and a proxy mode, built from one `httpserver.c` via `#ifdefs`. Starter PARTs run 1–7 *(index/tasks: Wayback 20251201183625–183639; background: 20250413083002 with a Dec-2025 revisit record)*.

1. **Socket (PART 1)** — finish `serve_forever`: `bind` to the requested port, `listen` with backlog 1024. Socket creation and `SO_REUSEADDR` are already in the starter. Checkpoint: `curl` reports "Empty reply from server".
2. **GET: files (PART 2)** — `handle_files_request` → `serve_file`: subdirectory paths work; `Content-Type` and a byte-accurate `Content-Length`; **raw `read`/`write` syscalls only — "implementations using `fread` or `fwrite` will not earn any credit"**; nonexistent file → `404 Not Found` (body optional). 404 is the **only** required error path.
3. **GET: directories (PART 3)** — `stat` + `S_ISDIR`/`S_ISREG`; directory with `index.html` → 200 + its contents (no trailing-slash assumption); without → an HTML listing of immediate children **plus a parent-directory link** (`http_format_href`/`http_format_index` helpers); nonexistent → 404; close the client socket before returning.
4. **Proxy (PART 4)** — `handle_proxy_request`: upstream connection setup (DNS, socket, connect) is provided. You shuttle bytes both directions, **supporting multiple requests/responses on one connection**, using **two pthreads (one per direction)**; the page explicitly says **"Do not use `select`, `fcntl`, or the like"** (a reversal of older semesters). When either side closes, close both.
5. **Servers (PARTs 5–7)** — `forkserver`: child serves and exits, parent does **not** wait, both sides close the fds they don't own; `threadserver`: a pthread per connection, **no join**; `poolserver`: exactly `--num-threads` concurrent clients — the main thread `wq_push`es accepted fds, pool threads block in `wq_pop` (`wq.h` provided).
6. **Performance** — `ab -n 500 -c 10` against all four variants, vary `n`, `c`, and the pool size. "You no longer have to answer these questions on Gradescope" — exploratory, **not** extra credit.

Usage (background page): `./httpserver --files www/ | --proxy host[:port] [--port 8000 --num-threads 5]`; argument parsing and accept/parse plumbing are starter-provided.

## 5.2 Starter map (`ORIG`)

`httpserver.c` (PART markers 1–7), `libhttp.{c,h}`, `wq.{c,h}`, `utlist.h`, `Makefile` (four binaries via `-D` defines), `www/index.html`, `www/my_documents/*` (test corpus, including binary files).

## 5.3 Review findings (former in-repo docs)

| Former claim | Verdict |
|---|---|
| “Create a TCP socket, set SO_REUSEADDR, bind, listen” as the socket task | **Contradicts** (half) — creation + SO_REUSEADDR are starter-provided; the task is bind+listen |
| “Reject `..` path traversal” as a task | **Unsupported** — no such requirement in the Fall 2025 handout |
| “Return 404 or 405”; tests for 403 | **Contradicts** — “we only expect you to support the 404 Not Found error message” |
| Performance labeled “(EC)” | **Contradicts** — not extra credit (and no longer submitted at all) |
| Proxy described without the pthread requirement or the select/fcntl ban | **Gap** — both are explicit and autograder-relevant |
| fork/thread/pool semantics | **Confirmed** |

## 5.4 Test contract

Smoke `10-build-variants`, `11-usage-no-args`, `12-no-stdio-file-io-static` (mirrors the fread/fwrite zero-credit rule). Gated `full/20..30`: header- and byte-exact GET checks, 404, index.html dispatch, listing + parent link, binary byte-exactness, all three concurrent variants, 10 parallel clients, and a proxy round-trip against a local upstream. TODO stubs `full/40..42` (same-connection keep-alive through the proxy, exact pool concurrency, the `ab` table for your REPORT). Recipes: Chapter 15.

# Chapter 6

# HW 3: Rust HTTP Server (`hw-http-rs`)

## 6.1 Official assignment (Fall 2025)

The Rust/Tokio twin of hw-http (no proxy, no fork/pool variants; adds a statistics module). The handout recommends working **locally** rather than in Docker for compile speed *(index: Wayback 20251031031755)*. Starter Parts 1–4:

1. **Part 1 — HTTP helpers** (`src/http.rs`): implement `end_headers` and `get_mime_type` (both `todo!("TODO: Part 1")`). Already provided: `parse_request`, `start_response`, `send_header`, `format_href`, `format_index`.
2. **Part 2 — listen** (`src/server.rs`, Wayback 20251210052834): bind `tokio::net::TcpListener` on `0.0.0.0:<port>`, accept in an infinite loop, `tokio::spawn` a task per connection calling `handle_socket`. Sanity check from the handout: a `curl` should crash the starter with `panicked at 'not yet implemented: TODO: Part 3'` — that means Part 2 works.
3. **Part 3 — GET handling** (Wayback 20251110124708): files — MIME type from the extension, byte-accurate `Content-Length`, **at most 1024 bytes of the file in memory at a time** (read/write loop), prepend `.` to the request path, `tokio::fs` only (never `std::fs`), 404 for missing (headers/body optional), and **never panic on internal errors** (`log::warn!` and continue — the autograder tests non-panic behavior). Directories — decide via `tokio::fs::metadata` (explicitly *not* via `File::open` errors: Windows/Linux divergence), `index.html` → 200 + contents, otherwise a listing of children **plus a parent link** (`format_href`), 404 for nonexistent; `Content-Type` required for listings (`Content-Length` optional there); use `Path`/`PathBuf`.
4. **Part 4 — statistics** (`src/stats.rs`, Wayback 20251110141348): `Stats::incr` via `HashMap::entry` (deliberately not thread-safe at first; named unit tests `incr_items`, `incr_items_four`, `items_sorted`), then share it as `Arc<tokio::sync::RwLock<Stats>>` (the handout names tokio's RwLock specifically, not `std::sync`'s), count every response's status code, and serve the counts at `/stats` (overriding any file there).

## 6.2 Starter map (`ORIG`)

`Cargo.toml` (tokio 1.19, clap 3.2, `panic = 'abort'`), `rust-toolchain.toml`, `src/{main,args,http,server,stats}.rs`, `src/tests/{mod,args,http,stats}.rs` (staff unit tests, `#[cfg(test)]`- gated), `www/`. On the pristine starter `cargo build` succeeds and **6 of the 12 staff unit tests fail** (the `todo!()`s in `end_headers`, `get_mime_type`, and `stats`) — the expected starting score.

## 6.3 Review findings (former in-repo docs)

| Former claim | Verdict |
|---|---|
| "`http.rs`: TODO `parse_request()`" | **Contradicts** — `parse_request` is given; Part 1 is `end_headers` + `get_mime_type` |
| "Stats: thread-safe with Mutex or atomics" | **Contradicts** — the handout specifies `Arc<tokio::sync::RwLock<…>>` |
| ≤1024-byte streaming limit; the `.`-prefix path rule | **Gap** — both are graded details |
| Server/dispatch description (spawn per connection; file/dir/404) | **Confirmed** |
| "6 of 12 unit tests fail on the starter" (validation note) | **Confirmed** |

## 6.4 Test contract

Smoke `10-cargo-build`, `11-smoke-suite` (`cargo test --test smoke`). Gated `full/20-unit-tests` (the 12 staff unit tests) and `full/21-behavior-suite` → `tests/behavior.rs`: black-box `#[ignore]` tests for 200 + Content-Length + exact body, MIME, 404, `/` → index.html, listing + parent link, `/stats` counts, and 10 concurrent clients, plus two `todo_*` stubs you implement (bounded-memory streaming; no-panic on malformed input). All behavior tests spawn the real binary via `CARGO_BIN_EXE_*` and always compile against the pristine `src/`. Recipes: Chapter 15.

# Chapter 7

# HW 4: Memory (`hw-memory`)

## 7.1 Official assignment (Fall 2025)

Two halves, both graded, both in `ORIG` — an **individual** assignment (explicitly: do not reuse Project-1 group code) *(index: Wayback 20250916012013; sbrk pages: 20251201141052)*.

### 7.1.1 Part A — the `sbrk` syscall in Pintos (`pintos/src`)

The provided mini-userspace already implements: the `do-nothing` fix, a `write(1)` syscall, single-file `open/read/write/close`, syscall argument validation, **and the user-side `malloc/calloc/realloc/free` library**. What's missing is the system call those library functions sit on:

- Implement `syscall_sbrk(intptr_t increment)` in `userprog/syscall.c`; wire `lib/syscall-nr.h`, `lib/user/syscall.c`, and the handler dispatch.
- Track two per-process values in `struct thread`: the heap start — a **page-aligned address after the last loadable ELF segment**, fixed at `load` time — and the segment break.
- `sbrk(0)` returns the current break. Growth that crosses page boundaries allocates with `palloc_get_page(PAL_USER | PAL_ZERO)` (zeroing is a protection requirement: reused frames must never leak other processes' data) and maps via `pagedir_set_page` (kernel virtual address, not a physical frame number). Shrinking deallocates pages that left the heap. Pages are touched only when the break crosses a boundary; memory between a non-aligned break and the next page boundary is legitimately accessible ("no man's land").
- On failure (e.g. user pool exhausted): return `(void*) -1` **with the break and heap unchanged** — roll back partial work.
- Explicitly out of scope: `brk`, heap hard limits, lazy allocation.

Official test command *(Wayback 20250408195100)*: `cd pintos/src/memory && make check` — 26 tests: `sbrk-{page,small,none,multi,zero,rv,large,mebi,fail-1,fail-2,dealloc,many,counter,oom-1,oom-2}` plus `malloc-{simple,free,fit,fail,merge-1,merge-2,null}` and `realloc-{1,2,3,null}` (the provided Pintos malloc starts passing once your `sbrk` works). `sbrk-oom-2` carries a 300 s timeout.

### 7.1.2 Part B — `mm_alloc`, a standalone allocator (`mm_alloc/`)

Implement `mm_malloc`/`mm_realloc`/`mm_free` in `mm_alloc.c` (headers fixed) on the **real** `sbrk` — libc `malloc`/`free`/`realloc` are banned *(Wayback 20250409013212, 20250408195136)*:

- linked-list blocks with `prev`/`next`/`free`/`size` metadata; first fit; split oversized blocks (or swallow the remainder when it can't hold a header); **zero-fill returned memory — "for grading purposes"**; `mm_malloc(0)` → NULL; allocation failure → NULL;

- `mm_free(NULL)` is a no-op; freeing **must coalesce** adjacent free blocks;
- `mm_realloc` edge cases: failure leaves the original intact; `(ptr, 0)` ≡ free → NULL; `(NULL, n)` ≡ malloc; `(NULL, 0)` → NULL; shrinking preserves the prefix.
- `mm_test.c` is a sanity harness the handout tells you to extend yourself.

Build: `cd mm_alloc && make` (`-Wall -Werror -Wextra`); the pristine skeleton (empty stubs) **does not compile** until you start — expected.

## 7.2 Review findings (former in-repo docs)

| Former claim | Verdict |
|---|---|
| Task structure “sbrk + library functions + standalone mm_alloc” | **Confirmed** — and worth stressing: an earlier suspicion (based on a Spring-2026 page summary) that `mm_alloc` was a non-official leftover was itself wrong; the Fall 2025 library-functions section *is* `mm_alloc` |
| “Implement malloc/calloc/realloc/free in `pintos/src/lib/user/`” | **Contradicts** — the starter provides the Pintos-side library; the kernel `sbrk` is the task |
| 26-test list + `make check` PATH recipe | **Confirmed** (matches `Make.tests`) |
| Alignment of returned pointers emphasized; zero-fill not mentioned | **Contradicts/Gap** — Fall 2025 grades zero-fill; it does not state an alignment requirement |

## 7.3 Test contract

Smoke `10-mm-alloc-starter-build` (honest SKIP while the skeleton can’t compile), `11-pintos-memory-builds` (kernel + test binaries build without qemu). Gated: `full/20-mm-test-runs`, `full/21-mm-behavior-driver` (compiles `tests/fixtures/mm_behavior.c` against your `mm_alloc.o`: zero-fill, `malloc(0)`, `free(NULL)`, first-fit reuse, realloc edge cases, coalescing — extend it as you go), `full/30-pintos-single-test` (`sbrk-zero` via `make …/sbrk-zero.result`), `full/31-pintos-make-check` (all 26; run with `CASE_TIMEOUT=3600`). TODO stubs `full/40..41` propose new Pintos user tests (no-man’s-land probe, multi-page shrink). Recipes: Chapter 15.

# Chapter 8

# HW 5: Map Reduce (`hw-map-reduce`)

## 8.1 Official assignment (Fall 2025)

A fault-tolerant MapReduce in C over SunRPC, loosely based on MIT 6.824's lab. Three graded components: the **RPC lab** (`lab-rpc`, 5%), a **checkpoint** (through the Tasks section, 5%, autograder build triggered manually), and the **final** (through Fault tolerance). Missing the checkpoint caps the assignment at 95%; missing the lab too caps it at 90%. Memory leaks are explicitly not graded *(index: Wayback 20251110143529)*.

**Everything except the coordinator is provided**: the complete RPC protocol (`rpc/rpc.x` — `SUBMIT_JOB`, `POLL_JOB`, `GET_TASK = 4`, `FINISH_TASK = 5` with all request/reply types), the full worker, the client (`submit` / `poll` / `process` with parsed args), the apps (`wc`, `grep`, `vertex-degree`), the length-delimited codec, `ihash`, the datasets, and a `words` oracle binary. The RPC server is **single-threaded — no synchronization needed** *(development page: Wayback 20251007154900)*.

You implement `coordinator/coordinator.c` (five TODO regions):

1. **`SUBMIT_JOB`** *(Wayback 20251201082246)* — unique job IDs counting up from 0, never reused; efficient lookup by ID; FCFS queue; `strdup`/ `memcpy` everything you keep (XDR frees RPC buffers after return); GLib (`GList`/`GHashTable`/`GQueue`) recommended; invalid app → error.
2. **`POLL_JOB`** — `done`/`failed`/`invalid_job_id`; an invalid ID must not crash the coordinator.
3. **`GET_TASK`** *(Wayback 20251007144023)* — assign `n_map` map tasks then `n_reduce` reduce tasks per job; **reduces only after all of that job's maps**; never assign a task to two workers; FCFS across jobs but **work-conserving** ("workers should not sit idle if there are available tasks" — idle workers take later jobs' map tasks); fill **every** reply field (a NULL string segfaults the XDR serializer); `wait = true` when nothing is available.
4. **`FINISH_TASK`** — bookkeeping; job completes when all tasks do; `success = false` → the job **fails immediately** (stop assigning; `POLL_JOB` → done+failed).
5. **Fault tolerance** *(Wayback 20251007154932)* — record each task's assignment time; a task incomplete after `TASK_TIMEOUT_SECS` is reassignable (the C variant has **no heartbeats** — timeout on the task itself). Completed tasks are never reassigned (outputs are on disk).

Run flow: `sudo service rpcbind start`, `./bin/mr-coordinator`, `./bin/mr-worker` (×N), `./bin/mr-client submit -a wc -o out -w -n 10 data/gutenberg/*`, then `./bin/mr-client process …`; validate `wc` against `./words FILE...` and `grep` against grep(1).

**Official autograder tests** (the development page documents all 21 with debugging hints — gold for local replication): `poll-bad-id`, `wc`, `grep`, `vertex-degree`, `map-parallel`, `map-parallel-add-worker`, `reduce-parallel`, `reduce-parallel-add-worker`, `mr-concurrent`, `coordinator-no-work`, `mr-queue-order`, `queue-map-multiple`, `mr-no-duplicates`, `map-crash`, `reduce-crash`, `rand-crash`, `crash-multi-job`, `crash-no-duplicates`, `submit-bad-app`, `map-error`, `reduce-error`.

## 8.2 Review findings (former in-repo docs)

| Former claim | Verdict |
|---|---|
| "`rpc/rpc.x`: TODO — declare RPC numbers and types" | **Contradicts** — the protocol "has already been provided for you" |
| "`worker/worker.c`: TODO — register, poll, execute"; "`client/client.c`: TODO — submit/process" | **Contradicts** — both fully provided (0 TODOs); the coordinator is the assignment |
| Fault tolerance via "heartbeats" and a "configurable retry count" | **Contradicts** — timeout-on-assigned-task; `success=false` fails the job immediately, no retry budget |
| Grading split 5/5/90 | **Confirmed** |
| "Schedule a reduce when all maps for that partition are done" | **Contradicts (detail)** — reduces wait for **all** of the job's maps |

## 8.3 Test contract

Smoke `10-build` (dependency-guarded: rpcgen/tirpc/glib), `11-binaries- exist`, `12-client-usage-no-hang`. Gated `full/20-wc-end-to-end` (coordinator + 3 workers vs the `words` oracle), `full/21-grep-end-to-end`, `full/22-submit-bad-app`, `full/23-vertex-degree-runs` — all require rpcbind and SKIP cleanly without it. TODO stubs `full/40..43` each mirror a named autograder test (`mr-queue-order`, `mr-no-duplicates`, `map-crash` with the handout's sleep-then-Ctrl-C drill, `poll-bad-id`). Recipes: Chapter 15.

# Chapter 9

# HW 5: Map Reduce, Rust (`hw-map-reduce-rs`)

> Snapshot caveat: the Fall 2025 window archived only the index and two background pages; all task-page citations below are complete April 2025 captures whose Fall 2025 presence is confirmed by the Fall index's navigation and CDX revisit records (see Chapter 1 and Appendix C).

## 9.1 Official assignment (Fall 2025)

The tonic/gRPC twin of the C MapReduce, same three components and grading caps (gRPC lab `lab-grpc-rs` 5%, checkpoint 5%, final). Local development strongly recommended; CMake ≥ 3.20 required for `tonic-build` *(index: Wayback 20251110141540)*.

**Provided**: every protocol buffer (`proto/coordinator.proto`, `proto/worker.proto`), the complete worker, the client, the apps, the codec, and an autograder driver binary. **You implement the coordinator** (`src/coordinator/mod.rs`) — and, as setup for the staff end-to-end test, replace the deliberate `todo!("Pick an input file")` in `src/tests/mod.rs` (until then a full `cargo test` does not even compile; `cargo test --test smoke` is unaffected).

1. **Register / Heartbeat** *(Wayback 20250425163928)* — unique, small, nonnegative worker IDs (IDs derive the workers' listening ports!); record each worker's last-heartbeat time. Unlike the C variant, **heartbeats are the liveness mechanism here**.
2. **SubmitJob / PollJob** — unique job IDs; FCFS queue with efficient ID lookup (`VecDeque` + `HashMap`); the handout prescribes **one** `Arc<tokio::sync::Mutex<CoordinatorState>>` around all mutable state and warns that staff won't debug multi-mutex designs. `PollJob` reports `done`/`failed`, and its `errors` field must carry worker-reported error messages **verbatim** (adding context is allowed).
3. **GetTask / FinishTask** (checkpoint boundary = passing through `mr-no-duplicates`) — "each worker should be assigned the first available task"; reduces become available only when their job's maps are all done; FCFS across jobs but work-conserving; the reply's `map_task_assignments` tells a reduce worker **which workers hold which map outputs — map output lives in worker memory**, not on disk.
4. **Fault tolerance** — a worker is dead when it hasn't heartbeated for `TASK_TIMEOUT_SECS` (slow ≠ dead — never reassign a live worker's task). Consequences of in-memory map output: a crashed worker's **completed map tasks must be re-executed** (with priority over later jobs' tasks, but without preempting running workers); completed reduce tasks stay done (on disk). `FailTask` with `retry = true` (reduce worker couldn't reach a map worker) → reassign; `retry = false` (unfixable: missing input, unwritable output, app error) → the job fails

immediately and the error string must reach the client via `PollJob`.

## 9.2 Review findings (former in-repo docs)

| Former claim | Verdict |
|---|---|
| "`proto/*.proto`: TODO — define gRPC services" | **Contradicts** — all protobufs are provided |
| Registration + heartbeat described as student tasks | **Confirmed** (correct for the Rust variant) |
| "Fail the job after N retries" | **Contradicts** — `retry=false` fails immediately; `retry=true` reassigns with no budget |
| Single-mutex guidance; ID→port constraint; in-memory map-output rerun rule; verbatim error propagation | **Gap** ×4 — all grading-relevant, all absent |
| The uncommitted edit that filled in `todo!("Pick an input file")` with a real path | **Contradicts the starter's intent** — that hole is the student's setup step; reverted |

## 9.3 Test contract

Smoke `10-cargo-build`, `11-smoke-suite`. Gated `full/20-staff-test-wc` (skips until the `todo!` is replaced, then `cargo test test_wc -- --ignored`) and `full/21-behavior-suite` → `tests/behavior.rs` (`e2e_wc_job_completes` spawns `mr-coordinator` + two `mr-workers` via `CARGO_BIN_EXE_*`, submits with `--wait=true`, post-processes, asserts non-empty plausible output) plus three `todo_*` stubs (FCFS order, crashed-worker map re-run, verbatim error propagation). Recipes: Chapter 15.

# Chapter 10

# RPC Lab (`lab-rpc`)

## 10.1 Official assignment (Fall 2025)

A guided `rpcgen`/SunRPC warm-up worth 5% of HW Map Reduce: build a tiny key/value store *(index: Wayback 20251202075625)*. Editable files: **`kv_store.x`, `server.c`, `kv_store_client.c` only** — `client.c` (the CLI with `example`/`echo`/`put`/`get` subcommands) is provided.

1. **Protocol** *(Wayback 20260131191049, Fall-2025-equivalent)* — extend the `KVSTORE` program in `kv_store.x` with `ECHO` (procedure number 2; keep numbers unique and sequential). `EXAMPLE(int) = 1` is the worked model.
2. **Server** *(Wayback 20251007154519)* — after the `.x` edit, `make` fails until you add `echo_1_svc` to `server.c`. Follow `example_1_svc`'s convention: the reply lives in a `static` variable (stack locals die at return); no `malloc` needed.
3. **Client** *(Wayback 20250916003454)* — implement `echo()` in `kv_store_client.c`: connect, call the generated `echo_1` stub, **`strdup` the returned string before `xdr_free((xdrproc_t) xdr_string, …)`**, destroy the client. It must actually round-trip through the server — the autograder checks.
4. **KV store** *(Wayback 20250916011936)* — add `PUT` and `GET`: multi-argument requests are wrapped in an XDR struct; keys and values are variable-length byte arrays via the provided `typedef char buf<>` and are **not necessarily NUL-terminated**. Server state: a global `GHashTable` storing `GBytes`, initialized in `main`; the RPC server is single-threaded, so no locking. (GET-on-missing-key behavior is left to you — define and document your contract.)

Environment: `sudo apt-get install rpcbind`, then `sudo service rpcbind start` after every container start; `make` runs `rpcgen` and produces `bin/server` + `bin/client`; `./bin/client example 1` → `2` works on the pristine starter.

## 10.2 Review findings (former in-repo docs)

| Former claim | Verdict |
|---|---|
| "`client.c`: TODO — add echo/put/get subcommands" | **Contradicts** — `client.c` is provided and outside the editable-files list; the student side is `kv_store_client.c` |
| "Store keys/values in any structure, e.g. GHashTable" | **Imprecise** — the handout directs GHashTable + GBytes specifically |
| Binary-safety of keys/values; the strdup-before-xdr_free trap | **Gap** ×2 |

| Former claim | Verdict |
|---|---|
| rpcgen flow, EXAMPLE-as-model, rpcbind requirement | **Confirmed** |

## 10.3 Test contract

Smoke `10-build`, `11-binaries-exist`, `12-client-no-args-no-hang` (all dependency-guarded). Gated `full/20-example-roundtrip` (passes on the pristine starter — your environment check), `full/21-echo-roundtrip`, `full/22-put-get-roundtrip`, `full/23-put-overwrite`. TODO stubs `full/40-binary-safe-keys` (needs a small C driver against `kv_store_client.h` — argv can't carry NUL bytes) and `full/41-get-missing-key` (assert your documented contract *and* that the server survives). Recipes: Chapter 15.

# Chapter 11

# gRPC Lab (`lab-grpc-rs`)

## 11.1 Official assignment (Fall 2025)

The tonic/protobuf twin of lab-rpc, worth 5% of HW Map Reduce (Rust). Local development recommended; CMake ≥ 3.20 *(index: Wayback 20251206133213)*. Editable files: **`proto/kv_store.proto`, `src/server.rs`, `src/client.rs` only** — `src/rpc/kv_store.rs` is regenerated on every build and must not be edited.

1. **Protocol buffers** *(Wayback 20251215030623)* — add an `Echo` RPC to the `KvStore` service with `EchoRequest`/`EchoReply` (the lab fixes the convention: RPC `Foo` ⇔ `FooRequest`/`FooReply`; sequential field numbers). The provided `Example` (input + 1) is the model.
2. **RPC implementation** *(Wayback 20251215030658)* — after the proto edit, `cargo build` errors until the `echo` trait method exists on the `KvStore` service in `src/server.rs` (`req.into_inner()`, wrap the reply in `Response::new`). Verify with `cargo check`.
3. **Issuing RPCs** *(Wayback 20251215030658)* — implement `echo()` in `src/client.rs` following `example()`; then with the server running, `./target/debug/client echo test` prints `test`.
4. **KV store** *(Wayback 20251215030658)* — add `Put` and `Get`: `bytes` fields (⇄ `Vec<u8>`); `Put` overwrites an existing key; `Get` on a missing key returns **exactly** `Err(Status::new(Code::NotFound, "Key does not exist."))`. State: a `HashMap` behind **`Arc<tokio::sync::RwLock<…>>`** — this server is async and multi-threaded, so unlike the SunRPC lab, synchronization is mandatory; add the field to the `KvStore` struct and initialize it in `start`. Working `Echo` + `Put` + `Get` = full autograder score.

`src/test.rs` ("Do not modify") is the staff acceptance suite (`test_example`, `test_echo`, `test_put`/`test_get`); the tests call your client helpers and need a live server on `127.0.0.1:10162` (`SERVER_ADDR` in `src/lib.rs`).

## 11.2 Review findings (former in-repo docs)

| Former claim | Verdict |
|---|---|
| "Declare `Example`, `Echo`, `Put`, `Get` RPCs" | **Imprecise** — `Example` is given; you add the other three |
| "Backed by `tokio::sync::Mutex<HashMap>`" | **Contradicts** — the handout specifies `tokio::sync::RwLock` |
| Exact NotFound contract (`"Key does not exist."`) and overwrite-on-duplicate | **Gap** ×2 — both autograder-checked |

| | |
|---|---|
| “`src/test.rs`: modify only after implementing” | **Contradicts** — the file says “Do not modify”; it is the acceptance suite |

## 11.3 Test contract

Smoke `10-cargo-build`, `11-smoke-suite`. Gated `full/20-example-roundtrip-cli` (passes on the pristine starter once the server binary is up — plumbing check), `full/21-staff-acceptance` (spawns `target/debug/server`, waits on 10162, runs the `src/test.rs` suite single-threaded — the autograder contract), `full/22-cli-echo-put-get`. TODO stub `full/40-notfound-exact-status` (assert both `Code::NotFound` and the exact message). Recipes: Chapter 15.

# Chapter 12

# Project Pregame (`proj-pregame`)

## 12.1 Official assignment (Fall 2025)

An **individual** Pintos warm-up: read the Pintos docs (overview, user programs, threads, processes, memory allocation, lists, development), then diagnose and fix why the simplest user program crashes *(index: Wayback 20251007151923)*. Deliverables *(Wayback 20251201130649)*: **17 written Gradescope questions** (reader-graded) plus a small kernel fix whose autograder contract is exactly three passing tests: **`do-nothing`, `do-not-much`, `stack-align-v`**.

1. **Find the faulting instruction** *(Wayback 20251201130601)* — in `src/userprog`: `make`, `make check`, observe **no tests pass** on the starter. Study `tests/userprog/do-nothing.c` (returns 162 = 0xa2), read the `.result` diff, then answer: the faulting virtual address and why it's inaccessible (cite Pintos macros), the faulting instruction's address, the disassembled function (`i386-objdump`), its C source (in `src/lib/user`), and a calling-convention explanation of why that instruction touches that address.
2. **Step through the crash** *(Wayback 20251201130709)* — from `src/userprog`: `FORCE_SIMULATOR=--bochs PINTOS_DEBUG=1 pintos-test do-nothing` (alias it as `pintos-debug`). Breakpoints `run_task` → `process_execute` → `start_process`; dump `struct thread`s (`dumplist &all_list thread allelem`), `print/x if_` after `load`, step the `asm volatile` into `iret` and explain the user-mode switch, `info registers`, `loadusersymbols tests/userprog/do-nothing`, and `btpagefault` at the fault. (7 questions.)
3. **Debug** *(Wayback 20250408174658, Fall-2025 revisit record)* — make `do-nothing` pass with a kernel-side change ("our staff solution solves this with a single-line change to `process.c`"; hacks explicitly OK — the real fix comes in Project Userprog). Then `do-not-much` (returns `argc`), then `stack-align-v` (returns `esp % 16`; the expected value is in `stack-align-v.ck`) — justify the final fix with explicit calling- convention offsets. Finish by stepping to `int $0x30`, printing `x/2xw $esp`, and relating it to `args[0]`/`args[1]` in `syscall_handler`. (5 questions + the push.)

## 12.2 Starter map (`ORIG`)

The full Pintos tree under `proj-pregame/src` (threads, userprog, lib, devices, filesys, vm stubs, `tests/userprog`, and `utils/` with `pintos`, `pintos-gdb`, `backtrace`). Tests build and run from `src/userprog` (`make`, `make check`, or `make build/tests/userprog/<t>.result`).

## 12.3 Review findings (former in-repo docs)

| Former claim | Verdict |
|---|---|
| “Find and fix a kernel bug; confirm with `cd src/threads && make check`” | **Contradicts** — the assignment lives in `src/userprog`; the deliverable tests are `do-nothing`, `do-not-much`, `stack-align-v`, not the threads suite |
| “Fix the bug in `init.c` (or elsewhere)” | **Misleading** — the handout points at a one-line `process.c` change |
| No mention of `do-not-much`/`stack-align-v` or the 17 Gradescope questions | **Gap** — most of the project’s graded surface |
| General gdb/bochs debugging advice | **Confirmed**/kept (Chapter 13, Appendix A) |

## 12.4 Test contract

Smoke `10-userprog-builds` (kernel + `do-nothing` binary compile without an emulator). Gated `full/20-do-nothing`, `full/21-do-not-much`, `full/22-stack-align-v` — one case per official autograder test, each run via `make build/tests/userprog/<t>.result` under qemu, each failing on the unfixed starter by design. The 17 written questions live in `proj-pregame/REPORT.md` as a fill-in table. Recipes: Chapter 15.

# Part II

# Local Testing Handbook

# Chapter 13

# Testing fundamentals

This chapter consolidates the techniques formerly spread across `docs/testing-tutorial.md`, `docs/cs162-homework-guides/testing-guide.md`, and the per-assignment `TESTING.md` files. Read it once; Chapter 15 then applies it assignment by assignment.

## 13.1 What a test is

A test answers exactly one question: *does this specific behavior work?* Two consequences: **one behavior per case** (when a case can fail for five reasons, a red result tells you almost nothing), and **the fixture is part of the question** — keep inputs small enough that you can hand-compute the expected output. If you can only obtain the expected output by running the program, you are observing, not testing.

Every case has four phases — arrange, act, assert, clean up — and cleanup must run on failure too. In this framework, cleanup is automatic: anything started with `spawn` or registered with `cleanup_add` is torn down when the case exits, and each case gets a private `$T_TMP` that the runner deletes.

A complete suite, per the old completeness standard (kept, it's good): builds from a clean checkout with one command; every required feature has a positive *and* a negative test; output compared by content, not just exit status; nondeterminism normalized; concurrency stressed repeatedly; hangs bounded by timeouts; every fixture has a reason; skips documented; reproducible from a fresh clone. Keep a feature-by-test coverage matrix per assignment (the REPORT.md test tables serve this purpose).

## 13.2 Fixtures and golden files

Prefer the handout's own examples as fixtures — they are the spec (`hw-intro/words/words.txt` ships byte-identical to the handout's example; hw-shell's redirection example is encoded as case `full/29`). For deterministic output, diff against a golden file (`assert_same_file`, `golden`); when the spec pins no order, normalize both sides (sort, `normalize_ws`, `strip_hex_addrs`) — and when the spec *does* pin an order (hw-intro's frequency sort), assert the order, never sort it away.

Always include failure-path tests: missing file, bad flag, malformed request, missing key, invalid app, port already bound. The standard bash shape is `run … ; [ "$RUN_STATUS" -ne 0 ] || t_fail …`. Crashes are distinguishable from clean failures: exit ≥ 128 means a signal.

## 13.3 Timeouts, servers, and ports

Anything that can block runs under `timeout` — in this framework `run` defaults to 10 s and the runner hard-caps a case at `CASE_TIMEOUT` (300 s default). Server tests use two patterns together: **ephemeral ports** (`pick_port`; never hard-code 8000) and **wait-for-ready** (`wait_tcp host port secs` polls at 100 ms; never `sleep 1` and hope). Spawn servers with `spawn` so they die with the case. HTTP checks are easiest through `curl -s --noproxy '*'` plus `-D` for headers or `-w '%{http_code}'` for the status.

## 13.4 Concurrency

Correct-looking output proves nothing about races. Three escalating tools: **stress loops** (run the same comparison 25–100×: `hw-list full/30-pwords-race-stress`), **race detectors** (`valgrind --tool=helgrind`, or ThreadSanitizer via `-fsanitize=thread`), and **adversarial scheduling** (the MapReduce handout's own trick: insert a `sleep` after a worker receives a task, then kill it — fault-tolerance paths only fire when you make them fire). For parallelism claims (hw-http's pool, pwords' fine-grained locking), assert **time**: wall-clock of the concurrent run must be well under the serialized sum.

## 13.5 Memory

For C assignments: `valgrind --leak-check=full --error-exitcode=1` on a small fixture, or compile a test driver with `-fsanitize=address`. Keep memcheck out of the default path (slow); give it its own case. Note the exceptions the handouts themselves make: hw-map-reduce explicitly does not grade leaks; hw-memory's Pintos side has its own `sbrk-*`/`malloc-*` tests for failure paths.

## 13.6 Signals need a pty

Job control depends on a controlling terminal: through a plain pipe, `tcsetpgrp` fails with `ENOTTY` and Ctrl-C never reaches a foreground group, so a pipe-based signal test cannot distinguish your bug from the harness's. Drive the shell under a pseudo-terminal:

```
import os, pty, signal, time
pid, fd = pty.fork()
if pid == 0:
    os.execvp('./shell', ['./shell'])
os.write(fd, b'sleep 30\n'); time.sleep(0.3)
os.killpg(os.tcgetpgrp(fd), signal.SIGINT)      # Ctrl-C the fg group
os.write(fd, b'/bin/echo alive\n'); time.sleep(0.3)
out = os.read(fd, 4096)                          # assert b'alive' in out
```

Use the pty only for signal/job-control cases (`hw-shell full/40..42, 50..51`); everything else is faster through pipes. Mind the race between delivering a signal and reading the pty — add a short sleep before asserting post-signal state.

## 13.7 Honest skipping

A test that "passes" on the unmodified starter when the starter cannot possibly satisfy it is worse than no test. This framework's protocol makes honesty cheap: exit 77 = SKIP (environment can't run it — `require_cmd`, `require_rpcbind`, `require_qemu`, `try_build`), exit 78 = TODO (a stub you still owe yourself), and `cases/full/` is gated behind `RUN_FULL=1` so the default run is green on

the pristine starter *without lying* — the gated cases then fail honestly until you implement the feature. Known intentional starter breakage (hw-intro `words`, `mm_alloc`, hw-map-reduce-rs's full `cargo test`) is encoded as SKIP-with-explanation, never "fixed".

## 13.8 Reading a failure, and TDD order

Read the diff first; shrink the fixture next; if results differ across runs, suspect a race; `git bisect run bash tests/run.sh <glob>` finds the breaking commit. Never widen an expectation to silence a failure you don't understand. Working order for each assignment: pick the next handout task → un-stub or write its `full/` case (red) → implement minimally (green) → refactor with the suite as your safety net. The TODO stubs shipped per assignment are exactly these red tests, pre-named after the official autograder list so your local suite converges on the real grading surface.

## 13.9 Smells

A case name that names no behavior; more than ~4 assertions per case; an expected value computed by the program under test; no timeout or cleanup; order-dependent cases; a flake you didn't investigate; "fixing" the test instead of the code.

# Chapter 14

# Framework reference

## 14.1 Layout

```
student0/
├── run_all_tests.sh          # walks all 11 assignments, summary table
├── Makefile                  # make test / test-full / clean
├── tests/
│   ├── lib.sh                # helper library (sourced by every case)
│   └── runner.sh             # discovery, outcome protocol, totals
└── <assignment>/
    ├── tests/run.sh          # 3-line wrapper → ../../tests/runner.sh
    ├── tests/cases/NN-name.sh        # always-run cases (smoke)
    ├── tests/cases/full/NN-name.sh   # gated cases (RUN_FULL=1)
    ├── tests/fixtures/…              # inputs, golden files, C drivers
    └── REPORT.md             # your lab report (fill as you go)
```

Rust crates additionally use cargo-native integration tests — `tests/smoke.rs` (always green on the starter), `tests/behavior.rs` (`#[ignore]`d black-box tests + `todo_*` stubs), `tests/common/mod.rs` (spawn/wait/HTTP helpers) — orchestrated from thin bash cases so the `run_all_tests.sh` summary covers them too. No original file (Makefiles, `src/`, `*.x`, `*.proto`) is modified by the framework.

## 14.2 Running

```
bash run_all_tests.sh                # smoke everything (green on pristine starter)
RUN_FULL=1 bash run_all_tests.sh     # + behavior tests (fail until implemented)
ONLY="hw-shell hw-http" bash run_all_tests.sh
bash hw-shell/tests/run.sh           # one assignment
bash hw-shell/tests/run.sh 'full/2*'  # name-glob filter
RUN_FULL=1 VERBOSE=1 bash hw-shell/tests/run.sh full/20-pwd-builtin
```

Environment knobs: `RUN_FULL=1` (run `cases/full/`), `VERBOSE=1` (stream case output), `KEEP_TMP=1` (keep per-case temp dirs), `CASE_TIMEOUT=N` (hard wall-clock per case; default 300 — raise to 3600 for `hw-memory full/31`).

## 14.3 Outcome protocol

A case is an executable bash script; its exit status is the verdict: **0 PASS · 77 SKIP · 78 TODO · 124 FAIL(timeout) · anything else FAIL.** SKIP/TODO/GATED never fail a run; the runner exits non-zero iff some case FAILed. Per-assignment output ends with a machine-readable `#totals pass=… fail=… skip=… todo=… gated=…` line, which `run_all_tests.sh` collects into its summary table.

## 14.4 Writing a case

```
#!/usr/bin/env bash
# Case: <one behavior>. Handout: Fall 2025 <hw>, docs/<page>.
source "${TESTLIB:?run via tests/run.sh}"
require_full                  # only for cases/full/ (honest when run directly)
build "$HW_DIR"               # cached per run; FAILs with the log tail
run -t 5 -i $'input\n' "$HW_DIR/prog" arg
assert_eq "$RUN_OUT" "expected"
```

Conventions: number cases (`10-` smoke, `20-` implemented behavior, `30-` stress, `40-` your TODO frontier, `50-` extra credit); cite the handout page in the header; one behavior per file; create files only under `$T_TMP` (or `tmpfile`); start servers with `spawn` + `wait_tcp`. To add a case: drop a new `.sh` into `cases/` or `cases/full/` — discovery is automatic, no registry to edit. To turn a TODO stub into a real test: replace its `t_todo "…"` line with arrange/act/assert, keeping the header citation.

## 14.5 `tests/lib.sh` API

| Group | Helpers |
|---|---|
| Outcomes | `t_fail msg` · `t_skip reason` (77) · `t_todo reason` (78) · `t_diag msg` |
| Assertions | `assert_eq got want [label]` · `assert_ne` · `assert_contains hay needle` · `assert_matches str ERE` · `assert_exit code cmd…` · `assert_file_exists` · `assert_executable` · `assert_same_file expected actual` |
| Execution | `run [-t secs] [-i stdin] cmd…` → `RUN_OUT`/`RUN_ERR`/`RUN_STATUS` (never fails by itself) · `must_run …` (asserts status 0, dumps output on failure) |
| Processes | `spawn cmd…` (→ `SPAWN_PID`, `SPAWN_LOG`; auto-killed) · `cleanup_add 'cmd'` (LIFO at exit) · `wait_tcp host port [secs]` · `pick_port` |
| Guards (→ SKIP) | `require_cmd name…` · `require_file path` · `require_rpcbind` · `require_qemu` · `require_full` |
| Builds | `build DIR [targets…]` (make; cached per run via `$T_SHARED` sentinels; FAIL + log tail) · `try_build DIR [targets…]` (returns 1 instead of failing; sets `TRY_BUILD_LOG` — for honest SKIPs) · `cargo_build DIR [args…]` |
| Text | `golden actual golden [filter]` · `normalize_ws` · `strip_hex_addrs` |

| Group | Helpers |
|---|---|
| Misc | `fixture name` → `$HW_DIR/tests/fixtures/name` · `tmpfile` |

Cases run with `set -u -o pipefail` (not `-e`): a failure is whatever you assert, plus any helper's explicit `t_fail`.

## 14.6 Rust crates

Black-box rules that keep `tests/behavior.rs` compiling against the pristine starter: spawn binaries via `env!("CARGO_BIN_EXE_<name>")`, never `use` crate internals; guard every spawned process with a `Drop`-based kill; run server-bound suites with `--test-threads=1` when they share a fixed address (lab-grpc-rs's 10162, map-reduce's coordinator). Selective runs: `cargo test --test behavior serve_index -- --ignored`.

## 14.7 Pintos pieces

`hw-memory/pintos` and `proj-pregame/src` build on the host (gcc multilib) — that is the smoke layer. Running tests needs `qemu-system-i386` (`require_qemu`); single tests go through `make build/tests/<suite>/<name>.result` from the relevant `src/<project>` directory with `utils/` on `PATH`; full suites via `make check` with a raised `CASE_TIMEOUT`. Test verdicts are written by Pintos' perl checkers (`.ck` files) — read the `.ck` to understand an expected-output mismatch.

## 14.8 Using the REPORT.md templates

Every assignment ships a tailored `REPORT.md`: an environment table, one Task-Log subsection per official handout task (with the evidence test cases pre-named), a test-results table matching the framework's case names, a debugging log, and — where the handout has written deliverables — a Gradescope answer table (hw-intro, hw-list, proj-pregame). Fill it as you work, not after: the debugging log ("symptom → hypothesis → fix → test that now catches it") is the habit the whole framework exists to build. Paste the `#totals` lines or the `run_all_tests.sh` summary into §Test Results before submitting anything.

# Chapter 15

# Per-assignment test recipes

Each section: what ships, what the pristine starter should show, and the roadmap from there to a complete suite shaped like the official grading surface. “Starter expectation” is what an honest run prints **before you write any assignment code**.

## 15.1 hw-intro

*Starter expectation:* smoke 3 pass + 1 SKIP (`13-words-starter-build` — `-Werror`); all `full/` gated. *Roadmap:* implement total count → `full/20..22` green; frequency → `full/23` (the header line is tolerated, the five `<count> <word>` lines are exact); error handling → `full/24`; limits → `full/25`. Then write `full/40` (a fixture of one-letter tokens — total must count only runs of length > 1), `full/41` (case folding: `Foo FOO foo` → `3 foo`), `full/42` (`ulimit -Ss/-Sn` cross-check; allow the documented negative process-limit quirk). Pitfalls: don’t use `rewind`/`fseek` (stdin breaks); remember `words -c` is the *default* mode per the handout examples (no flag).

## 15.2 hw-list

*Starter expectation:* smoke 3 pass (stubs link!); `full/` gated. *Roadmap:* `word_count_l.c` → `full/20`; `pwords.c` + `word_count_p.c` → `full/21..23`, then keep `full/30` (25× stress) green. Your frontier: `full/40` — prove fine-grained locking with wall-clock (two large inputs: threaded time must approach max, not sum, of per-file times); `full/41` — helgrind clean on the fixtures; `full/42` — a 20-line driver compiled in the case that sorts with `less_word` to prove comparator genericity. Pitfalls: compare against the `words` oracle with `diff` (its format is `%8d\t%s` — don’t hand-roll expectations); one mutex around `count_words` will pass parity tests and still lose points — that’s why `full/40` exists.

## 15.3 hw-shell

*Starter expectation:* smoke 4 pass (including the `execvp` grep); `full/` gated. *Roadmap:* built-ins → `20..21`; exec → `22`, `28`; PATH → `23`; redirection → `24..25`, `29`; pipes → `26..27`. Then the pty frontier (`40..42`) using Chapter 13’s harness — assert: shell survives SIGINT and still answers; SIGTSTP stops only the job (`/proc/<pid>/stat` state `T`); child PGID ≠ shell PGID and TPGID == child PGID. EC: `50` (background `&` returns the prompt early — wall-clock it), `51` (`fg`/`bg`). Pitfalls: prompt text pollutes exact-equality — strip or match loosely; use absolute paths (`/bin/echo`) so PATH tests stay independent of exec tests; keep `test.sh` (staff script) as a second opinion.

## 15.4 hw-http

*Starter expectation:* smoke 3 pass; `full/` gated (server can't accept until PART 1). *Roadmap:* PART 1 makes `full/20` reach "Empty reply" stage (curl exit ≠ refused); PARTs 2–3 → `20..25`; PARTs 5–7 → `26..29`; PART 4 → `30`. Frontier: `40` (two GETs over one proxied connection — raw fd via `exec 3<>/dev/tcp/…` or python socket), `41` (pool concurrency: N+2 slow clients against `--num-threads N`, time first bytes), `42` (the `ab` table → REPORT §2.5). Pitfalls: `--noproxy '*'` on every curl (corporate proxies break localhost); Content-Length must equal *body* bytes (verify with `wc -c`, not `strlen` of a text read); test binaries (`.jpg`) with `cmp`, never text diff.

## 15.5 hw-http-rs

*Starter expectation:* smoke 2 pass; `full/20` fails 6 of 12 staff unit tests (by design), `full/21` panics at "TODO: Part 3" — both documented. *Roadmap:* Part 1 → staff `http/stats` unit tests; Part 2 → behavior `ten_concurrent_clients` reaches the Part-3 panic; Part 3 → `serve_index…`, `missing_file_404`, `root_serves_index_html`, `dir_listing…`; Part 4 → staff `incr_items*/items_sorted` + behavior `stats_url_reports_counts`. Frontier: implement `todo_streaming_uses_bounded_memory` (generate a ~200 MB file in a tempdir, serve it, sample `VmHWM` from `/proc/<pid>/status`) and `todo_no_panic_on_malformed_input` (garbage bytes, missing blank line, huge request line — server must keep serving). Pitfall: run server-bound tests `--test-threads=1` or give each test its own port (the helpers already pick ports).

## 15.6 hw-memory

*Starter expectation:* smoke = 1 SKIP (mm_alloc can't compile yet) + 1 pass (Pintos builds); `full/` gated. *Roadmap:* implement `mm_alloc` → `full/20` (the handout's `mm_test`) and `full/21` (the spec driver — **extend `tests/fixtures/mm_behavior.c`** with: write-every-byte, many small alloc/free cycles, shrink-preserves- prefix, reuse-after-free patterns). Implement `sbrk` → `full/30` (`sbrk-zero` alone, fast loop) → `full/31` (`CASE_TIMEOUT=3600`, expect `All 26 tests passed.`). Frontier `40..41`: author new Pintos user tests (copy a `sbrk-*` test + its `.ck`, add to `Make.tests`) for no-man's-land and multi-page shrink behavior. Pitfalls: run memcheck (`-fsanitize= address`) on the *driver*, not `mm_test` (dlopen confuses ASan leak reports); zero-fill is graded — test it on *reused* blocks, not just fresh sbrk memory.

## 15.7 hw-map-reduce

*Starter expectation:* smoke 3 pass (deps installed) or honest SKIPs naming the missing package; `full/` SKIPs without rpcbind. *Roadmap (= checkpoint order):* SUBMIT_JOB/POLL_JOB → `full/22`; GET_TASK/ FINISH_TASK → `full/20` (wc vs the `words` oracle), `full/21`, `full/23`; fault tolerance → implement `full/42` with the handout's own drill (sleep after `get_task_1`, SIGKILL the worker, job must still finish correctly). Then mirror the remaining named autograder tests — `full/40` (`mr-queue-order`: one worker, two jobs, log GET_TASK order), `full/41` (`mr-no-duplicates`: log (job,task,reduce) triples, assert uniqueness), `full/43` (`poll-bad-id`: a 30-line RPC client cloned from client.c's poll path). The development page's per-test hint list is your debugging index. Pitfalls: never share an output dir between jobs (intermediate-file races); kill workers with SIGKILL, not SIGTERM, to simulate crashes; vary worker count and `n_reduce` — several autograder tests only bite at non-default sizes.

## 15.8 hw-map-reduce-rs

*Starter expectation:* smoke 2 pass; `full/20` SKIPs until you fill the `todo!("Pick an input file");` `full/21` fails at the first unimplemented RPC. *Roadmap:* Register/Heartbeat (log-verified) → SubmitJob/PollJob → GetTask/FinishTask → `full/20` + behavior `e2e_wc_job_completes` green = checkpoint shape; fault tolerance → implement the three `todo_*` behavior tests (FCFS order via coordinator logs; crashed-worker map re-run; verbatim error propagation). Pitfalls: keep ONE mutex (the handout's explicit guidance); worker IDs must stay small (they become ports — a UUID "works" until the bind fails); `--wait=true` (clap bool-with-value), not bare `--wait`.

## 15.9 lab-rpc

*Starter expectation:* smoke 3 pass (deps present); `full/20` passes even on the starter once rpcbind runs — use it to validate the environment before blaming your code. *Roadmap:* ECHO in `.x` → server → client → `full/21`; PUT/GET → `full/22..23`; then `full/40` (binary keys via a C driver against `kv_store_client.h` — model it on hw-memory's driver case) and `full/41` (your documented missing-key contract + server-survives). Pitfalls: every new RPC needs a fresh number; rerun `make` after each `.x` edit (stubs regenerate); `RPC: Authentication error` usually means a stale server instance is still registered — kill it and restart.

## 15.10 lab-grpc-rs

*Starter expectation:* smoke 2 pass; `full/20` passes on the starter (Example is implemented); `full/21..22` fail until Echo/Put/Get exist. *Roadmap:* Echo proto → server (`cargo check`) → client → Put/Get with the RwLock'd map → `full/21` (the staff `src/test.rs` suite — the autograder contract) and `full/22`. Frontier `full/40`: assert the exact `Code::NotFound` + `"Key does not exist."` contract. Pitfalls: kill stray `server` processes between runs (fixed port 10162); never edit `src/rpc/kv_store.rs`; remember Put must overwrite.

## 15.11 proj-pregame

*Starter expectation:* smoke 1 pass; `full/20..22` all FAIL — the handout opens with "observe that currently no tests pass". *Roadmap:* the three gated cases *are* the autograder; make them pass in order (do-nothing → do-not-much → stack-align-v) while drafting the 17 written answers in REPORT.md as you step through GDB/bochs. No TODO stubs: the remaining depth here is written analysis, not more automation. Pitfalls: run tests from `src/userprog` with `utils/` on PATH; use `FORCE_SIMULATOR=--bochs` only for the debug walkthrough (qemu for plain runs); when a `.result` confuses you, read the matching `.ck`.

# Part III

# Appendices

# Appendix A

# Self-check questions and answers

Curated from the repository's former `STUDY.md` files — kept where correct, corrected where they contradicted the Fall 2025 handouts, with fresh answers. Try to answer before reading.

## A.1 hw-intro

**Q1. Why does the handout insist on the soft limits in `limits.c`, and which limit governs an actual `open()` call?** Soft limits are the enforced ones — a process hits `EMFILE` at the *soft* fd limit; the hard limit is just the ceiling up to which an unprivileged process may raise its own soft limit. Printing hard limits would "work" yet describe nothing about runtime behavior.

**Q2. Running `./words` on `/dev/urandom` never terminates. Why, and what makes it safe?** The loop reads until `EOF`, and `/dev/urandom` never returns one; the program is correct by its spec (read-until-EOF is exactly what stdin mode requires). Safety comes from the caller (`timeout 5 ./words < /dev/urandom`) — which is precisely why every framework `run` carries a timeout.

**Q3. `map` prints different addresses each run. What feature causes that, and how do you get stable addresses for a debugging session?** ASLR randomizes stack/heap/mmap bases per exec. The course sidesteps it by pinning the environment — `i386-gdb-map.sh` runs gdb under `env -i` with a fixed PATH precisely so the addresses (and your Gradescope answers) are reproducible; locally, `setarch -R` or gdb's default disable-randomization achieves the same.

**Q4. Why must case-folding happen before insertion into the word list rather than at print time?** The list is the data structure of record: `AaA` and `aaa` must land in *one* node with count 2 (the handout's fixture demands `2 aaa`). Folding at print time leaves two nodes whose counts the sort then orders separately — the output is wrong even though every word "looks" lowercase.

## A.2 hw-list

**Q1. The Pintos list is intrusive — the `list_elem` lives inside your struct. What does that buy?** No per-node allocation: linking a `word_count_t` costs zero extra mallocs, removal needs no search for a wrapper node, and the same element can sit in several lists via several embedded elems. The price: the container can't own nodes of arbitrary type, and you must use `list_entry` to get from elem back to struct.

**Q2. Two threads `add_word(list, "hello")` concurrently; "hello" isn't present yet. Give a losing interleaving.** Both threads scan and find no node; both allocate a node for "hello";

both insert. The list now has two "hello" nodes each with count 1 — `pwords` prints a duplicate row or a short count vs `words`. This is exactly what `full/30`'s 25× stress run is fishing for.

**Q3. Why does the handout demand fine-grained locking when one global lock around `count_words` also produces correct output?** Because the point of `pwords` is *concurrent file processing*: a global lock serializes the reads and makes the threaded program a slower `lwords`. Staff explicitly reserve the right to deduct for it. Correctness tests can't see the difference — a timing test (full/40) can.

**Q4. helgrind flags a race in `pwords` but output is always right. Fix it?** Yes. "Always right so far" is the definition of a latent race; helgrind is reporting an unsynchronized access pair that some scheduler interleaving can break. Either prove the access is logically protected (then annotate), or lock it. Never ship a known data race because today's runs pass.

## A.3 hw-shell

**Q1. Why must `cd` and `exit` be built-ins?** They mutate the *shell's own* process state (cwd; existence). A forked `/usr/bin/cd` would chdir its child copy and exit — parent unchanged. Anything that must alter shell state (also `fg`/`bg`, variable exports) has to run in-process.

**Q2. The shell never `waitpid`s its children. Immediate vs long-term symptom?** Immediately: nothing visible — prompts keep coming. Long-term: every exited child stays a zombie holding a process-table slot until the table fills and `fork` fails with `EAGAIN`. Foreground children must be waited for anyway (the prompt should return *after* the command), background ones reaped via `WNOHANG` polling or `SIGCHLD`.

**Q3. Why must `>`/`<` tokens never reach the child's argv?** `execv` doesn't interpret shell syntax: `ls > out` becomes `argv = {"ls", ">", "out"}` and `ls` happily lists files named `>` and `out`. The shell opens the file, `dup2`s it over fd 0/1 between fork and exec, and passes a *stripped* argv. (The handout's `wc < in > out` example output betrays this exact bug: an extra filename appears after the counts.)

**Q4. In `A | B | C`, why must every child *and the shell* close unused pipe ends?** A reader gets `EOF` only when *all* write-end descriptors are closed. Leaked write ends (in siblings or the shell) leave `B`/`C` blocked forever — the classic full-pipeline hang the 3-stage test provokes; leaked read ends turn a dead reader into `SIGPIPE`-less infinite buffering.

**Q5. Why does the shell ignore `SIGINT` yet Ctrl-C still kills the foreground child?** Ctrl-C is delivered by the kernel to the *foreground process group* of the terminal. The shell parks each job in its own group (`setpgid`) and gives that group the terminal (`tcsetpgrp`), so the signal goes to the job, not the shell. The child must reset handlers to `SIG_DFL` after fork, since ignored dispositions survive `exec`.

## A.4 hw-http

**Q1. `forkserver`: what must each side close after `fork`?** The child closes the *listening* socket (it serves one connection and exits); the parent closes the *connection* socket (the child owns it). Skipping the parent's close leaks an fd per request and keeps connections half-open; skipping the child's close holds the listening port across crashes.

**Q2. Why is a condition variable (inside `wq`) the right pool primitive rather than spinning on `trylock`?** Pool threads must *block* when the queue is empty — a spin loop burns a core per idle thread and still races. `wq_pop`'s cond-wait sleeps until a push signals, giving exactly the "serve at most N concurrently, queue the rest" semantics `poolserver` is graded on.

**Q3. The proxy must support multiple requests per connection. Why does that force the two-thread design the handout prescribes?** You can't know which side talks next (pipelined

requests, chunked responses, server push of the next reply). One thread per direction turns "who's ready?" into two blocking `read→write` loops — and the handout explicitly bans `select`/`fcntl` here. Teardown: when either side EOFs, close both sockets so the twin thread's blocked call returns.

**Q4. Serving a 1 GB file with a 1024-byte read/write loop saturates CPU. What's the grown-up fix and why is it out of scope here?** `sendfile(2)` (or `splice`) moves file→socket bytes in-kernel, no user-space copies. The assignment *requires* the raw `read`/`write` loop precisely to make you confront short reads/writes — the pedagogy is the copy loop.

## A.5 hw-http-rs

**Q1. Tokio task vs OS thread — why does it matter for this server?** A task is a cheap state machine scheduled cooperatively on a small thread pool; an `.await` yields the thread to other tasks. Ten thousand idle connections cost ten thousand small futures, not ten thousand stacks. The price: blocking calls (std::fs, CPU loops) stall a whole worker thread — hence the handout's tokio::fs mandate.

**Q2. Why `Arc<RwLock<Stats>>` rather than `Rc<RefCell<Stats>>`?** `Rc`/`RefCell` are single-threaded by construction (`!Send`/`!Sync`) — the compiler rejects them across `tokio::spawn`. `Arc` makes the refcount atomic; the lock makes mutation exclusive; `RwLock` (vs `Mutex`) lets many `/stats` readers proceed concurrently while writers serialize.

**Q3. `panic = 'abort'` is set for this crate. What does that change?** A panic kills the whole process instead of unwinding one task — there is no catching it at the runtime boundary. That's why the handout grades "your server does not panic upon an internal error": with abort semantics, one malformed request would take down every connection.

**Q4. Why does `Stats::items()` sort internally instead of leaving it to callers?** The sorted order *is* the contract (`items_sorted` is a named staff test). Pushing it to callers makes every consumer re-implement (and some forget), and HashMap iteration order would leak nondeterminism into output the autograder compares.

## A.6 hw-memory

**Q1. Why does Pintos need a kernel `sbrk` at all — couldn't user code mmap?** This Pintos has no `mmap`; the only way to get more user-mapped pages is for the kernel to install them. `sbrk` is the minimal contract: one moving boundary, kernel allocates/maps (or unmaps) the crossed pages. The provided user-side `malloc` is built on exactly that contract.

**Q2. `sbrk(4096)` returns, the write to the returned address faults. Which bug?** The syscall moved the break without mapping the page: missing `palloc_get_page(PAL_USER|PAL_ZERO)` + `pagedir_set_page` for the crossed boundary (or mapping at the wrong address — e.g. passing a physical frame number instead of the kernel virtual address). `sbrk-small`/`sbrk-page` catch precisely this.

**Q3. Why is coalescing essential in `mm_alloc`, and the symptom of skipping it?** Splitting creates ever-smaller neighbors; without merging on free, a sequence like alloc/free of varied sizes fragments the list until a request fits in *no* block even though total free space is ample — `sbrk` then grows the heap forever. Symptom: memory footprint climbs across a steady workload; `malloc-merge-1/2` and the spec driver's 96-byte check fail.

**Q4. Why zero-fill rather than hand back reused bytes as-is?** For the standalone allocator it's the graded contract (and it hides use-after-free bugs from *other* tests less). In the kernel it's security: a reused frame may hold another process's data; mapping it unwiped leaks that memory across protection domains — which is why `PAL_ZERO` appears in the kernel recipe too.

## A.7 hw-map-reduce (C)

**Q1. Why must all of a job's map tasks finish before any of its reduce tasks start?** A reduce task must see *every* key-value pair hashed to its partition; maps still running may yet emit pairs for it. Starting early yields silently incomplete groups — the handout's wc-test hints list "error opening intermediate files / reading while writing" as the observable crash modes.

**Q2. Why is one shared hash function (`ihash % n_reduce`) load-bearing?** Partitioning is an agreement: every map worker must send the same key to the same bucket index. Two hash functions split a key across two reduce tasks — both emit a count for the same word, and the merged output has duplicate keys. Cross-machine determinism is the whole trick.

**Q3. The C coordinator detects crashes by task timeout, not heartbeats (the Rust variant uses heartbeats). Why are completed tasks *not* reassigned here, while the Rust variant re-runs a dead worker's completed maps?** In the C design, map output is written to files on shared disk — a finished task's value survives its worker. In the Rust design, map output stays in the worker's memory and is served to reducers over RPC — a dead worker takes its completed map output with it, so those maps must run again. Storage placement dictates the fault-tolerance policy.

**Q4. Why is `process` (output post-processing) client-side?** The coordinator schedules; it never touches data (the `coordinator-no-work` autograder test enforces this). Reducers leave `mr-out-*` files; presentation-layer merging is the consumer's job — keeping the control plane off the data path is the same design choice real systems make.

## A.8 hw-map-reduce-rs

**Q1. Why does the *coordinator* assign worker IDs rather than letting workers pick UUIDs?** IDs must be unique *and small*: the handout derives each worker's listening port from its ID (u16!). A central counter guarantees both and gives the coordinator a stable index for heartbeat and assignment state. A UUID "works" until it has to become a port.

**Q2. Why `tokio::time::sleep` instead of `std::thread::sleep` in an async fn?** `std::thread::sleep` blocks the runtime worker thread, starving every task scheduled on it; `tokio::time::sleep().await` parks just the task. With a handful of worker threads and many heartbeat loops, the blocking version visibly stalls the cluster.

**Q3. What actually tells the coordinator a worker died?** Nothing at the transport level — gRPC calls just stop arriving. Death is a *policy*: no heartbeat for `TASK_TIMEOUT_SECS`. The handout adds the other half: slow-but-alive workers must NOT be declared dead, so the timeout compares against the last heartbeat, never task duration.

**Q4. Why segregate `tests/smoke.rs` from in-crate unit tests?** Files under `tests/` compile as *external* crates: they see only the public API and the built binaries — exactly an autograder's view, and they can't rot into reaching private internals. In-crate `#[cfg(test)]` modules test internals. The framework leans on the external kind so behavior tests always compile against the pristine starter.

## A.9 lab-rpc

**Q1. What does `rpcgen` generate from one `.x` declaration?** From `int EXAMPLE(int) = 1` it emits the XDR (de)serializers (`kv_store_xdr.c`), a client stub `example_1` that marshals, sends, and unmarshals, a server dispatch loop that routes procedure 1 to your `example_1_svc`, and the shared header tying them together. You write only the two endpoint functions.

**Q2. What problem does `rpcbind` solve?** SunRPC services bind ephemeral ports; `rpcbind` is the well-known registry (port 111) mapping (program, version) → current port. The server registers at startup; clients ask rpcbind first. No rpcbind, no discovery — hence the lab's "sudo service rpcbind start" ritual and this framework's `require_rpcbind` SKIP guard.

**Q3. Why does returning a pointer to a stack local from `echo_1_svc` "work in testing" then corrupt under load?** The stub serializes *after* your function returns, reading memory whose frame is gone. With one quiet client the stale frame often survives long enough; interleaved calls and deeper stacks scribble over it. Hence the `static` result convention (and its corollary: SunRPC handlers are single-flight; state needs no lock here).

**Q4. Keys "are not necessarily NUL-terminated" — what does that change?** Everything string-shaped becomes length+bytes (`buf{len,val}`): copy with `memcpy`/`GBytes`, compare with length-aware equality, and never `strlen`/ `strcpy` a key. Your CLI can't even express such keys — which is why the binary-safety test needs a C driver, not `./bin/client`.

## A.10 lab-grpc-rs

**Q1. What does protobuf give over XDR for schema evolution?** Tagged, optional-by-default fields: unknown tags are skipped, so old and new binaries interoperate as long as tag numbers stay stable — add fields without recompiling the world. XDR is positional and version-locked by the program/version pair; evolution means a new version block.

**Q2. Why is stub generation at build time (`build.rs`) rather than committed?** The generated code is a pure function of the `.proto` and the tonic version; committing it invites drift (edit the proto, forget the regen — or worse, hand-edit the generated file, which the lab explicitly forbids). `cargo build` regenerating guarantees coherence; the cost is needing CMake/protoc at build time.

**Q3. `Status::not_found(...)` vs `Ok(None)` for a missing key?** The RPC contract has exactly two channels: a reply message or a `Status`. An in-band `None` forces every caller to learn your sentinel and frees no one from handling transport errors anyway. The graded contract uses the error channel with a precise code and message — assert both in your test.

**Q4. Why does this KV server need a lock when the SunRPC one didn't?** tonic serves each call as a task on a multi-threaded runtime: two `Put`s can mutate the map concurrently. SunRPC's dispatcher is single-threaded — serial by construction. Same abstraction, different execution model, and the synchronization requirement follows the execution model.

## A.11 proj-pregame

**Q1. "Find the faulting instruction" — what's the actual path from crash to source line?** The `.result`/pintos output gives the faulting EIP and address. Disassemble the *user* binary (`i386-objdump -d build/tests/userprog/do-nothing`), locate that EIP, name the function, then map to C in `do-nothing.c` or `src/lib/user`. (Note: the crash is in userspace just before `main` runs — not "early in boot" as the old study notes claimed.)

**Q2. Why i386 (32-bit) Pintos in 2025?** Pedagogy: segmented/paged 32-bit x86 has a two-level page table you can draw, an `iret`-based mode switch you can single-step, and a small register file — the concepts transfer, the bookkeeping stays tractable. QEMU's user/system emulation makes the host architecture irrelevant.

**Q3. Why does the `iret` in `start_process` "return" into user mode?** `if_` is a hand-built interrupt frame: it contains the user `%eip`, user `%esp`, and segment selectors with CPL 3, plus EFLAGS. `iret` pops exactly that layout and atomically loads it — privilege drops because the popped CS selector's RPL is 3. The kernel never "calls" user code; it *returns* to it.

**Q4. What should `stack-align-v` return, and why?** `esp % 16` at the measuring point must be the value its `.ck` expects (the SysV i386 convention: the stack is 16-byte aligned *at the call boundary*, so `%esp ≡ 12 (mod 16)` right after the call pushes the return address — read the `.ck` and justify with the offsets). It exists because a fix that merely subtracts "enough" bytes for do-nothing/do-not-much can still misalign the stack.

# Appendix B

# Validation record

## B.1 Prior validation (2026-05-10, absorbed from the former `validation-report.md`)

A scratch-copy implementation pass on this WSL workspace established that the toolchain can carry every assignment end-to-end:

- Toolchain observed: cargo/rustc 1.95.0, rpcgen, libtirpc headers, qemu-system-i386, glib-2.0 via pkg-config.
- Completed-and-tested in scratch: hw-intro words; hw-list lwords/pwords; hw-shell built-ins/exec/redirection/pipes; hw-http file/dir/404 across all four server modes; hw-memory `mm_alloc`; hw-memory Pintos suite — **"All 26 tests passed."**
- Builds verified: hw-map-reduce (after installing tirpc + glib).
- Confirmed starter-state failures (expected): hw-http-rs `cargo test` — 6 of 12 staff unit tests fail on the pristine skeleton; hw-map-reduce-rs full `cargo test` does not compile (`todo!("Pick an input file")`).
- Real mistakes the scratch tests caught (kept as evidence the suite has teeth): hw-list output-spacing mismatch vs the oracle; a misaligned `mm_alloc` header layout; hw-http's accept-loop bug surfacing under curl.

## B.2 Final framework run (2026-06-10, this repository state)

Smoke suite on the pristine starter (`bash run_all_tests.sh`):

```
assignment         result totals
----------         ------ ------
hw-intro           PASS   pass=3 fail=0 skip=1 todo=0 gated=9
hw-list            PASS   pass=3 fail=0 skip=0 todo=0 gated=8
hw-shell           PASS   pass=4 fail=0 skip=0 todo=0 gated=15
hw-http            PASS   pass=3 fail=0 skip=0 todo=0 gated=14
hw-http-rs         PASS   pass=2 fail=0 skip=0 todo=0 gated=2
hw-memory          PASS   pass=1 fail=0 skip=1 todo=0 gated=6
hw-map-reduce      PASS   pass=3 fail=0 skip=0 todo=0 gated=8
hw-map-reduce-rs   PASS   pass=2 fail=0 skip=0 todo=0 gated=2
lab-rpc            PASS   pass=3 fail=0 skip=0 todo=0 gated=6
lab-grpc-rs        PASS   pass=2 fail=0 skip=0 todo=0 gated=4
proj-pregame       PASS   pass=1 fail=0 skip=0 todo=0 gated=3
```

```
ALL SUITES PASSED
```

Interpretation: all suites PASS; the SKIPs are the documented honest ones (`hw-intro` words and `hw-memory` mm_alloc cannot compile until implemented; dependency guards report anything missing on your host). Gated (`RUN_FULL=1`) cases fail on the pristine starter by design — that is the red end of your red→green workflow.

# Appendix C

# Wayback snapshot index

Every handout citation in Part I resolves as `https://web.archive.org/web/<timestamp>/<URL>`. Pages marked *(out-of-window)* were not captured inside 2025-08..2026-01; the nearest capture was used and cross-checked as described in Chapter 1.

## C.1 hw-http-rs

| Timestamp | Page | Note |
|---|---|---|
| 20251031031755 | (index) | |
| 20251007145447 | docs/background | |
| 20251110124708 | docs/requests | |
| 20250805013223 | docs/rust | |
| 20251210052834 | docs/socket | |
| 20251110141348 | docs/stats | |

## C.2 hw-http

| Timestamp | Page | Note |
|---|---|---|
| 20251201183630 | (index) | |
| 20250413083002 | docs/background | out-of-window |
| 20251201183630 | docs/submission | |
| 20250408221948 | docs/tasks | out-of-window |
| 20251201183625 | docs/tasks/get_request | |
| 20251201183630 | docs/tasks/performance | |
| 20251201183630 | docs/tasks/proxy | |
| 20251201183630 | docs/tasks/servers | |
| 20251201183630 | docs/tasks/socket | |

## C.3 hw-intro

| Timestamp | Page | Note |
|---|---|---|
| 20251215013051 | (index) | |
| 20251215163710 | docs/counting-words | |
| 20251215163710 | docs/executable | |
| 20251215163642 | docs/gdb | |
| 20251110130406 | docs/setup | |
| 20251110130607 | docs/setup/autograder | |
| 20240225153352 | docs/setup/cloud-setup | out-of-window |
| 20250805005630 | docs/setup/code-docker | |
| 20240417113242 | docs/setup/code-vm | out-of-window |
| 20251215163702 | docs/setup/code | |
| 20251008233309 | docs/setup/docker-workspace | |
| 20251215163702 | docs/setup/git-debug | |
| 20251215163702 | docs/setup/github | |
| 20240524192049 | docs/setup/virtualbox-setup | out-of-window |
| 20240524210009 | docs/setup/vm-config | out-of-window |
| 20240524194428 | docs/setup/vm | out-of-window |
| 20251210050249 | docs/setup/workspace-config | |
| 20251215163710 | docs/submission | |
| 20251215163710 | docs/tools | |
| 20251215163712 | docs/user-limits | |

## C.4 hw-list

| Timestamp | Page | Note |
|---|---|---|
| 20251008233309 | (index) | |
| 20251202115726 | docs/lwords | |
| 20260131004901 | docs/pthread | |
| 20251202115733 | docs/pwords | |
| 20251202115733 | docs/submission | |

## C.5 hw-map-reduce-rs

| Timestamp | Page | Note |
|---|---|---|
| 20251110141540 | (index) | |
| 20250425163918 | docs/background | out-of-window |
| 20250425163918 | docs/background/code | out-of-window |
| 20250715002932 | docs/background/development | out-of-window |
| 20251007150717 | docs/background/general-tips | |
| 20250425163918 | docs/background/grpc | out-of-window |
| 20250425163909 | docs/background/map-reduce | out-of-window |
| 20250425163929 | docs/conclusion | out-of-window |
| 20250425163919 | docs/example | out-of-window |
| 20250425163928 | docs/fault-tolerance | out-of-window |
| 20250425163928 | docs/job-submission | out-of-window |
| 20250425163928 | docs/tasks | out-of-window |
| 20250425163928 | docs/worker-registration | out-of-window |

## C.6 hw-map-reduce

| Timestamp | Page | Note |
|---|---|---|
| 20251110143529 | (index) | |
| 20251110125431 | docs/background | |
| 20251110140427 | docs/background/code | |
| 20251007154900 | docs/background/development | |
| 20251110133343 | docs/background/general_tips | |
| 20250916011303 | docs/background/map-reduce | |
| 20250916010043 | docs/background/rpc | |
| 20251110125242 | docs/conclusion | |
| 20251110131247 | docs/example | |
| 20251007154932 | docs/fault-tolerance | |
| 20251201082246 | docs/job-submission | |
| 20251007144023 | docs/tasks | |

## C.7 hw-memory

| Timestamp | Page | Note |
|---|---|---|
| 20250916012013 | (index) | |
| 20250409013212 | docs/library-functions/introduction | out-of-window |
| 20250408195136 | docs/library-functions/memory-allocator | out-of-window |
| 20250409001322 | docs/library_functions | out-of-window |
| 20251208214226 | docs/sbrk | |
| 20251201141052 | docs/sbrk/additional-information | |
| 20250408195100 | docs/sbrk/introduction | out-of-window |
| 20251201141052 | docs/sbrk/pages | |
| 20251201141052 | docs/sbrk/syscall-implementation | |
| 20251201141052 | docs/submission | |

## C.8 hw-shell

| Timestamp | Page | Note |
|---|---|---|
| 20251110134217 | (index) | |
| 20260126044745 | docs/directory-commands | |
| 20260126044755 | docs/foreground-background | |
| 20251106180536 | docs/foreground-background/background-processes | |
| 20251106180612 | docs/foreground-background/switching | |
| 20251106180548 | docs/path-resolution | |
| 20251106180524 | docs/pipes | |
| 20251106180548 | docs/program-execution | |

| Timestamp | Page | Note |
|---|---|---|
| 20251106180538 | docs/redirection | |
| 20260126044745 | docs/signal-handling | |
| 20251106180551 | docs/signal-handling/foreground-terminal | |
| 20251106180549 | docs/signal-handling/process-groups | |
| 20260126044744 | docs/signal-handling/shells-in-shells | |
| 20251106180545 | docs/signal-handling/signals | |
| 20251106180600 | docs/signal-handling/task | |
| 20260126044758 | docs/submission | |
| 20251106180552 | docs/testing | |

## C.9 lab-grpc-rs

| Timestamp | Page | Note |
|---|---|---|
| 20251206133213 | (index) | |
| 20251215030658 | docs/background | |
| 20251215030658 | docs/issuing-rpcs | |
| 20251215030658 | docs/kv-store | |
| 20251215030623 | docs/protocol-buffers | |
| 20251215030658 | docs/rpc-implementation | |

## C.10 lab-rpc

| Timestamp | Page | Note |
|---|---|---|
| 20251202075625 | (index) | |
| 20250916003454 | docs/issuing-rpcs | |
| 20250916011936 | docs/kv-store | |
| 20260131191049 | docs/protocol-specification | |
| 20251007154519 | docs/rpc-implementation | |

## C.11 proj-pregame

| Timestamp | Page | Note |
|---|---|---|
| 20251007151923 | (index) | |
| 20251201130649 | docs/deliverables | |
| 20251201130546 | docs/tasks | |
| 20250408174658 | docs/tasks/debug | out-of-window |
| 20251201130601 | docs/tasks/faulting_instruction | |
| 20251201130709 | docs/tasks/step_through | |